

Проект по дисциплине:
«Анализ изображений и компьютерное зрение»
FrameReader

Команда «Пришли пить кофе»

15 июня 2025 г.

Содержание

1	Введение	1
1.1	Описание проблемы	1
1.2	Зачем это нужно (потенциальные пользователи, кейсы применения)	1
1.3	Постановка задачи	1
1.4	Стек технологий	2
1.5	Этапы работы	2
2	Обзор существующих решений	3
2.1	Общие подходы и алгоритмы к OCR в видео	3
2.2	Анализ рынка и конкурентная среда	3
2.2.1	Основные категории конкурентов	3
2.2.2	Позиционирование проекта FrameReader	6
2.2.3	Сравнительный анализ: преимущества и недостатки FrameReader	6
2.2.4	Итоговый вывод по конкурентному анализу	8
2.3	Обоснование выбора методов в проекте FrameReader	8
3	Подготовка данных	8
3.1	Описание выбранного датасета	8
3.2	Предобработка данных	9
3.3	Визуализация ключевых аспектов данных	10
4	Разработка модели	10
4.1	Используемые алгоритмы и архитектуры	10
4.1.1	Детекция текста: YOLOv12	10
4.1.2	Распознавание текста (OCR): TrOCR	11
4.1.3	Распознавание текста (OCR): Donut	13
4.2	Обоснование выбора методов	13
4.3	Гиперпараметры и стратегия обучения	13
4.3.1	YOLOv12	13
4.3.2	TrOCR	15
4.3.3	Donut	15
5	Реализация и обучение	16
5.1	Описание кода и окружения	16
5.1.1	Окружение и фреймворки	16
5.1.2	Структура кода и скрипты обучения	17
5.2	Процесс обучения	17
5.2.1	YOLOv12	17
5.2.2	TrOCR	17
5.2.3	Donut	20

5.3	Метрики для детекции текста (YOLO)	20
5.4	Метрики для распознавания текста (TrOCR, Donut)	24
6	Развертывание и демонстрация	24
6.1	Использование в продакшене (Сервер инференса)	25
6.2	API и интерфейсы	25
6.2.1	API сервера инференса	25
6.2.2	API Бэкенда	26
6.3	Компоненты системы	26
6.3.1	Бэкенд (Server)	26
6.4	Демонстрация	27
7	Выводы и перспективы	27
7.1	Основные результаты работы	27
7.2	Нерешенные проблемы	27
7.3	Планы масштабирования	28

1 Введение

1.1 Описание проблемы

Наш проект FrameReader направлен на решение актуальной задачи автоматического извлечения и распознавания текстовой информации из видеопотоков. В современном мире объем видеоконтента постоянно растет, и ручной анализ каждого кадра на предмет наличия текста становится неэффективным и трудозатратным. Мы видим острую потребность в автоматизации этого процесса.

1.2 Зачем это нужно (потенциальные пользователи, кейсы применения)

Автоматическое извлечение текста из видео является критически важным для множества прикладных задач. Мы выделяем следующие ключевые кейсы применения и потенциальных пользователей:

- **Медиа-компании и контент-платформы:** Для индексации видеоконтента, автоматического создания субтитров, поиска по тексту внутри видео, мониторинга упоминаний брендов или ключевых слов в новостных сюжетах и рекламных материалах.
- **Маркетинговые агентства:** Для анализа эффективности рекламных кампаний, отслеживания размещения логотипов и текстовых сообщений в видеорекламе конкурентов.
- **Службы безопасности и видеонаблюдения:** Для автоматического распознавания номерных знаков, вывесок, предупреждающих надписей в видеопотоках с камер наблюдения.
- **Разработчики систем видеоаналитики:** Для интеграции функционала OCR в более сложные системы анализа поведения, событий или объектов в видео.
- **Образовательные учреждения:** Для индексации лекций и вебинаров, позволяя студентам быстро находить нужную информацию по тексту на слайдах или доске.

Наш проект предоставляет мощный инструмент для автоматизации этих процессов, значительно сокращая время и ресурсы, необходимые для анализа видеоконтента.

1.3 Постановка задачи

Основная задача, которую мы ставим перед проектом FrameReader, заключается в разработке комплексного и высокоэффективного решения для:

1. **Детекции текстовых областей на кадрах видео:** Точное определение местоположения и размеров текстовых блоков.

2. **Распознавания текста (OCR) из выделенных областей:** Преобразование изображений текстовых блоков в машиночитаемый текст.
3. **Предоставления удобного интерфейса для взаимодействия с системой и просмотра результатов:** Разработка интуитивно понятного пользовательского интерфейса для загрузки видео, мониторинга процесса обработки и визуализации полученных данных.
4. **Обеспечения масштабируемости и производительности решения для обработки больших объемов данных:** Создание архитектуры, способной эффективно обрабатывать значительные объемы видеоконтента с минимальными задержками.

Для решения этой задачи мы используем современные методы компьютерного зрения и машинного обучения. В качестве данных для обучения моделей мы активно используем датасет **RusTitW**, содержащий изображения русского текста, что позволяет нам достигать высокой точности для нашей целевой аудитории.

1.4 Стек технологий

Мы построили проект FrameReader на современном и эффективном стеке технологий, который обеспечивает гибкость, производительность и масштабируемость каждого компонента системы.

Ветка	Используемые технологии
Server	Python, FastAPI, WebSocket, MySQL
Client	React, Chakra UI, Axios, Rutube-player
Train	YOLOv12 (детекция текста), Donut (OCR), TrOCR (OCR)
Triton Server	NVIDIA Triton Inference Server, Ray Serve, Docker

1.5 Этапы работы

Работа над проектом FrameReader была структурирована и выполнена в соответствии со следующими основными этапами:

1. **Подготовка данных:** Мы осуществили сбор, тщательный анализ, предобработку и аугментацию датасета, чтобы обеспечить высокое качество обучающих данных.
2. **Разработка моделей:** Мы выбрали оптимальные архитектуры и провели настройку гиперпараметров для достижения наилучших результатов.
3. **Реализация и обучение:** Мы разработали код для обучения моделей и провели серию экспериментов для их оптимизации.

4. **Оценка качества моделей:** Мы рассчитали ключевые метрики и провели детальный анализ ошибок для каждой модели.
5. **Развертывание системы:** Мы настроили сервер инференса, разработали бэкенд и фронтенд, обеспечив их бесперебойное взаимодействие.
6. **Тестирование и демонстрация:** Мы провели всестороннее тестирование комплексного решения и подготовили демонстрационные материалы для наглядной презентации его возможностей.

2 Обзор существующих решений

2.1 Общие подходы и алгоритмы к OCR в видео

Для детекции и распознавания текста в видео существует множество подходов. Традиционные методы, такие как комбинация алгоритмов обработки изображений (например, MSER для выделения текстовых областей) и классических OCR-движков (например, Tesseract), часто страдают от низкой точности и производительности в условиях реального видео (размытие, движение, различные шрифты и освещение).

Мы сосредоточились на современных подходах, основанных на глубоком обучении. Для детекции текста широко используются архитектуры, такие как YOLO (You Only Look Once), SSD (Single Shot MultiBox Detector) и Faster R-CNN. Для распознавания текста (OCR) популярны модели на основе трансформеров, такие как TrOCR и Donut, которые показывают высокую точность даже на сложных изображениях.

2.2 Анализ рынка и конкурентная среда

Наш проект FrameReader решает задачу извлечения и распознавания текста из видеопотоков и функционирует в активно развивающейся области компьютерного зрения. На рынке существует ряд компаний, платформ и технологий, предлагающих аналогичные или пересекающиеся решения.

2.2.1 Основные категории конкурентов

1. Крупные облачные провайдеры Эти компании предлагают мощные и масштабируемые API-сервисы для OCR, включая функционал для видео. Их решения часто являются «черным ящиком» с точки зрения используемых моделей, но обеспечивают высокую надежность и простоту интеграции.

- **Google Cloud Vision AI:**

- **Описание:** Предоставляет API для анализа изображений и видео, включая детекцию и распознавание текста. Использует предобученные модели машинного обучения.

- **Конкурентное преимущество:** Высокая масштабируемость, надежность, интеграция с другими сервисами Google Cloud.
- **Аналог FrameReader:** Прямой конкурент в части предоставления OCR-функционала для видео.
- **Amazon Rekognition:**
 - **Описание:** Сервис компьютерного зрения от AWS, предлагающий детекцию текста в изображениях и видео.
 - **Конкурентное преимущество:** Глубокая интеграция в экосистему AWS, масштабируемость.
 - **Аналог FrameReader:** Аналогичен Google Cloud Vision в контексте OCR из видео.
- **Azure Cognitive Services (Computer Vision):**
 - **Описание:** Набор API для компьютерного зрения от Microsoft Azure, включающий OCR для печатного и рукописного текста в изображениях и видео.
 - **Конкурентное преимущество:** Интеграция с Azure, поддержка различных языков.
 - **Аналог FrameReader:** Аналогичен другим облачным решениям.

2. Специализированные платформы и инструменты Эти компании фокусируются на предоставлении более специализированных решений или инструментов для разработчиков в области компьютерного зрения и OCR.

- **Sieve:**
 - **Описание:** Платформа, которая занимается сравнением и предоставлением лучших методов OCR на видео, используя современные AI-модели (например, Gemini, Florence 2).
 - **Конкурентное преимущество:** Фокус на передовых моделях, возможно, более гибкие настройки для специфических задач.
 - **Аналог FrameReader:** Прямой конкурент, предлагающий готовые решения для видео-OCR.
- **Roboflow:**
 - **Описание:** Платформа для создания, аннотирования, обучения и развертывания моделей компьютерного зрения. Предоставляет инструменты для работы с OCR на видео, включая возможность аннотирования видеок кадров.
 - **Конкурентное преимущество:** Комплексный цикл разработки моделей, удобные инструменты для разметки данных.

- **Аналог FrameReader:** Конкурент в части обучения и развертывания моделей, а также предоставления инструментов для работы с видео.

- **Panopto:**

- **Описание:** Платформа для управления видеоконтентом, которая включает функции «умного» поиска по видео, используя ASR (автоматическое распознавание речи) и OCR для индексации.
- **Конкурентное преимущество:** Комплексное решение для корпоративного видео, интеграция ASR и OCR для поиска.
- **Аналог FrameReader:** Конкурент в более широком смысле, если FrameReader будет развиваться в сторону индексации и поиска по видео.

- **DENKnet (IDS Imaging):**

- **Описание:** Специализированное программное обеспечение для OCR с использованием AI, ориентированное на промышленные и специфические задачи.
- **Конкурентное преимущество:** Высокая точность для узкоспециализированных задач, кастомизация.
- **Аналог FrameReader:** Конкурент в нишевых применениях OCR.

3. Открытые библиотеки и фреймворки (для самостоятельной реализации)

Многие разработчики создают собственные решения, используя доступные библиотеки.

- **Tesseract OCR:**

- **Описание:** Один из самых популярных открытых OCR-движков. Может использоваться для распознавания текста на отдельных кадрах видео.
- **Конкурентное преимущество:** Бесплатность, гибкость, широкое сообщество.
- **Аналог FrameReader:** Базовый инструмент, который может быть частью более сложного решения, как в случае с tesseract.js в нашей клиентской части FrameReader.

- **OpenCV, FFmpeg:**

- **Описание:** Библиотеки для обработки изображений и видео, используемые для извлечения кадров, предобработки и других операций.
- **Конкурентное преимущество:** Фундаментальные инструменты для работы с медиа.
- **Аналог FrameReader:** Мы используем подобные инструменты как компоненты в нашей системе.

- **PyTorch, TensorFlow, Hugging Face Transformers, Ultralytics YOLO:**

- **Описание:** Фреймворки и библиотеки для машинного обучения, позволяющие разрабатывать и обучать собственные модели детекции и распознавания текста.
- **Конкурентное преимущество:** Полный контроль над моделями, возможность глубокой кастомизации.
- **Аналог FrameReader:** Это те самые инструменты, на которых построен FrameReader.

2.2.2 Позиционирование проекта FrameReader

FrameReader, с его архитектурой, включающей обучение кастомных моделей, использование Triton Server для инференса, а также собственный бэкенд и фронтенд, позиционируется как комплексное, кастомизируемое решение для OCR из видео с акцентом на русский язык. Ключевые конкурентные преимущества FrameReader заключаются в:

- Оптимизации под русский язык: Мы проводим специализированное обучение моделей на русскоязычных данных.
- Гибкости развертывания: Мы обеспечиваем возможность развертывания на собственном оборудовании с использованием Triton Server.
- Контроле над всем пайплайном: Мы контролируем весь процесс от обучения моделей до пользовательского интерфейса.
- Возможности интеграции: Мы предоставляем API для легкой интеграции в другие системы.

Таким образом, FrameReader занимает нишу между полностью облачными «черными ящиками» и полностью самописными решениями на базе открытых библиотек, предлагая баланс между готовым функционалом и возможностью глубокой кастомизации.

2.2.3 Сравнительный анализ: преимущества и недостатки FrameReader

Благодаря нашей специализации и архитектуре, FrameReader занимает уникальное положение на рынке решений для оптического распознавания текста (OCR) из видео.

Преимущества FrameReader:

- **Глубокая кастомизация и оптимизация под русский язык:** Мы обучаем и оптимизируем модели (YOLO, Donut, TrOCR) специально на русскоязычных данных (например, RusTitW). Это позволяет нам достичь значительно более высокой точности и релевантности для русского текста по сравнению с универсальными моделями. Большинство крупных облачных OCR-сервисов предлагают общие модели, которые могут не обеспечивать оптимального качества для русского языка.

- **Полный контроль над пайплайном (от обучения до развертывания):** FrameReader охватывает весь жизненный цикл моделей машинного обучения: от подготовки данных и обучения до оптимизации (TensorRT, квантизация), развертывания (NVIDIA Triton Inference Server, Ray Serve) и предоставления пользовательского интерфейса. Это дает нам полный контроль над каждым этапом. Облачные API ограничивают прозрачность и контроль.
- **Высокая производительность инференса:** Использование NVIDIA Triton Inference Server, Ray Serve и наша оптимизация моделей (TensorRT, квантизация) обеспечивают высокую пропускную способность и низкую задержку. Это критично для обработки больших объемов видео или работы в реальном времени. Open-source решения могут быть менее оптимизированы, а облачные сервисы дорогими при больших объемах.
- **Гибкость развертывания (On-Premise):** Мы обеспечиваем возможность развертывания всего решения на собственной инфраструктуре (on-premise) с использованием Docker. Это важно для организаций со строгими требованиями к безопасности данных или желающих избежать зависимости от облачных провайдеров.
- **Комплексное решение с открытым исходным кодом (частично):** FrameReader предоставляет достаточно открытых компонентов для понимания внутренней работы, адаптации или расширения, что не всегда возможно с проприетарными решениями.

Недостатки FrameReader:

- **Сложность развертывания и поддержки:** Развертывание и поддержка такого комплексного решения, как FrameReader, требует значительных технических знаний, опыта в DevOps и ML, а также вычислительных ресурсов. Это не «коробочное» решение. Облачные сервисы предлагают более простой процесс интеграции.
- **Отсутствие готовой инфраструктуры и SLA:** В настоящее время FrameReader не поставляется с готовой, управляемой инфраструктурой, гарантированной доступностью (SLA) и круглосуточной поддержкой. Эти аспекты ложатся на команду, развертывающую решение. Облачные сервисы это гарантируют.
- **Высокие первоначальные затраты на разработку и обучение:** Разработка наших кастомных моделей и создание всей инфраструктуры требует значительных первоначальных инвестиций времени и ресурсов. Готовые API позволяют начать работу быстро с оплатой по мере использования.
- **Ограниченная мультиязычность «из коробки»:** Хотя FrameReader оптимизирован под русский язык, поддержка других языков потребует с нашей стороны дополнительного обучения и адаптации моделей. Многие облачные решения предлагают широкую мультиязычную поддержку сразу.

- **Меньшая известность и экосистема:** Будучи относительно новым проектом, FrameReader пока не имеет такой широкой экосистемы, сообщества и готовых интеграций, как крупные игроки рынка. Крупные платформы имеют обширную документацию и активное сообщество.

2.2.4 Итоговый вывод по конкурентному анализу

FrameReader идеально подходит для компаний или проектов, которым требуется высокоточный и специализированный OCR для русского языка с возможностью глубокой кастомизации и развертывания на собственной инфраструктуре. Мы предлагаем мощный и гибкий инструментарий для тех, кто готов инвестировать в развертывание, поддержку и дальнейшее развитие нашего решения.

Для пользователей, которым нужна максимально простая интеграция, готовое решение «из коробки» и не критична специфическая оптимизация под русский язык или полный контроль над моделями, облачные API или специализированные платформы могут быть более предпочтительными.

2.3 Обоснование выбора методов в проекте FrameReader

Мы выбрали YOLOv12 для детекции текста, так как его высокая скорость и точность критичны для обработки видео в реальном времени. Для OCR мы остановили свой выбор на моделях Donut и TrOCR благодаря их способности эффективно работать с различными типами текста и документов, а также их возможностям для оптимизации под инференс. TrOCR хорошо зарекомендовал себя в задачах end-to-end OCR, в то время как Donut предлагает гибкость для понимания документов и может быть адаптирован для извлечения структурированной информации, что мы рассматриваем как перспективное направление для развития FrameReader.

3 Подготовка данных

3.1 Описание выбранного датасета

Для обучения моделей детекции и распознавания текста в нашем проекте FrameReader мы используем датасет [RusTitW \(Russian Language Visual Text Recognition\)](#). Этот датасет, разработанный Игорем Марковым, Сергеем Нестеруком, Андреем Кузнецовым и Денисом Димитровым (подробнее в [arXiv:2303.16531](#)), содержит изображения русского текста в естественных условиях (in-the-wild), что полностью соответствует основной задаче нашего проекта. Данные в RusTitW разделены на обучающую (train) и тестовую (test) выборки, каждая из которых, в свою очередь, содержит как реальные, так и синтетические (synth) примеры. Для удобства использования, каждая папка с данными сопровождается файлом `info.csv`, имеющим единый формат для всех частей датасета. Оригинальные метки и информация также сохранены и доступны в файлах `info_raw.csv` или `json_*.json`.

Стоит отметить, что в датасете присутствуют дубликаты изображений (например, одинаковые изображения с разным разрешением) и некоторое пересечение изображений между обучающей и тестовой выборками, что унаследовано из оригинальных источников данных.

Изначально, метки в датасете RusTitW представлены в формате JSON, описывающем ограничивающие рамки (bounding boxes) и соответствующий им текст. Пример такой метки:

```
[[{'left': 0.10259433962264151,
  'top': 0,
  'width': 0.4056603773584906,
  'height': 0.9303675048355899,
  'label': 'ALL you NEED\nis 20 SECONDS\nof Insane',
  'shape': 'rectangle'},
... % другие рамки
]]
```

Здесь `left`, `top`, `width`, `height` — относительные координаты и размеры рамки, а `label` — текст внутри нее.

3.2 Предобработка данных

Процесс предобработки данных в нашем проекте включает несколько ключевых этапов:

- **Нормализация:** Приведение изображений к стандартному размеру, например, 640x640 пикселей для модели детекции YOLO и 384x384 пикселей для моделей распознавания Donut и TrOCR.
- **Очистка:** Удаление некорректных или плохо аннотированных данных для повышения качества обучающей выборки.
- **Аугментация:** Применение различных преобразований к изображениям (повороты, масштабирование, изменение яркости/контрастности, добавление шума). Это позволяет увеличить разнообразие обучающей выборки и повысить устойчивость моделей к вариациям во входных данных, что способствует лучшей генерализации моделей на новые данные.
- **Конвертация формата для модели Donut:** Поскольку модель Donut требует специфическую структуру данных для обучения, мы выполняем конвертацию датасета RusTitW в требуемый формат. Это включает организацию данных в следующую иерархию директорий:

```
dataset_name /
+-- test /
|   +-- metadata.jsonl
|   +-- {image_path0}
```

```

|    L-- ...
+-- train/
|    +-- metadata.jsonl
|    +-- {image_path0}
|    L-- ...
L-- validation/
+-- metadata.jsonl
+-- {image_path0}
L-- ...

```

Каждый файл `metadata.jsonl` содержит строки в формате JSON, где каждая строка описывает одно изображение и его разметку:

```

{"file_name": "{image_path0}", "ground_truth": "{\\"gt_parse
  \": {ground_truth_parse}, ... }"}

```

Для задачи чтения текста (Text Reading Task), которая также является задачей предварительного обучения модели Donut, поле `gt_parse` имеет вид:

```

|{"text_sequence" : "word1 word2 word3 ... "}|

```

Для генерации синтетических данных в этом формате также может быть использован инструмент SynthDoG, описанный в репозитории Donut.

3.3 Визуализация ключевых аспектов данных

Для лучшего понимания характера данных, используемых в нашем проекте, ниже представлены примеры изображений из датасета RusTitW. Мы показываем как синтетические примеры, так и изображения из реальных условий.

4 Разработка модели

4.1 Используемые алгоритмы и архитектуры

В проекте FrameReader используются две основные категории моделей: для детекции текста и для распознавания текста.

4.1.1 Детекция текста: YOLOv12

Для детекции текстовых областей используется модель YOLOv12. YOLO (You Only Look Once) — это семейство моделей для обнаружения объектов в реальном времени. YOLOv12 является одной из последних итераций, предлагающей улучшенную точность и скорость.

Принцип работы YOLO: YOLO делит входное изображение на сетку. Каждая ячейка сетки предсказывает ограничивающие рамки (bounding boxes) и вероятности классов



Рис. 1: Примеры синтетических изображений из датасета RusTitW, используемых для обучения моделей.

для объектов, центры которых попадают в эту ячейку. Одновременно предсказываются координаты рамки (x, y, w, h) , где (x, y) — центр рамки, w — ширина, h — высота, а также вероятность наличия объекта (objectness score) и вероятности классов.

4.1.2 Распознавание текста (OCR): TrOCR

TrOCR (Transformer-based Optical Character Recognition) — это модель, основанная на архитектуре трансформера, разработанная Microsoft для оптического распознавания символов. Она использует подход "end-to-end" то есть принимает изображение на вход и выдает текст на выход, без необходимости в промежуточных шагах, таких как сегментация символов.

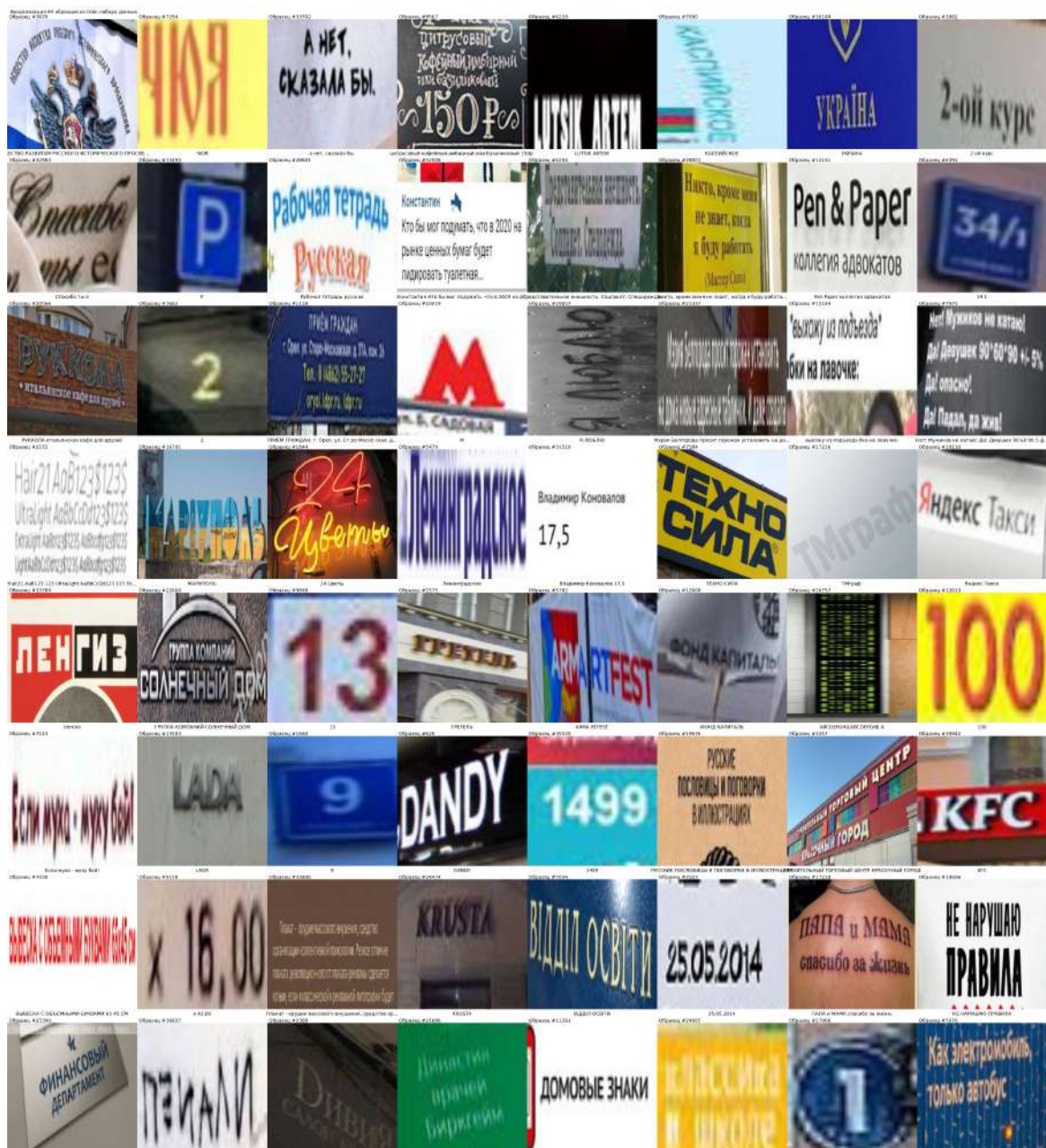


Рис. 2: Примеры реальных изображений ("in-the-wild") из датасета RusTitW, демонстрирующие разнообразие текстовых сцен.

Рис. 3: Пример детекции текста с помощью YOLOv12

Архитектура TrOCR: TrOCR состоит из двух основных частей:

- **Визуальный трансформер-энкодер:** Преобразует входное изображение в последовательность признаков.
- **Текстовый трансформер-декодер:** Генерирует последовательность символов (текст) на основе признаков, полученных от энкодера.

Модель обучается как seq2seq задача, где вход — изображение, а выход — последовательность символов.

4.1.3 Распознавание текста (OCR): Donut

Donut (Document Understanding Transformer) — это еще одна трансформер-модель, разработанная для понимания документов. В отличие от TrOCR, Donut ориентирован на более сложные задачи, такие как извлечение информации из форм или таблиц, но также может использоваться для общего OCR.

Особенности Donut:

- **Не требует OCR-движка:** Donut напрямую преобразует изображение в структурированный вывод (например, JSON), минуя промежуточный этап распознавания текста.
- **Гибкость:** Может быть адаптирован для различных задач понимания документов.

4.2 Обоснование выбора методов

Выбор конкретных архитектур (YOLOv12, TrOCR, Donut) подробно обсуждался в разделе ?? (Обзор существующих решений). Ключевыми факторами являются высокая производительность YOLOv12 для детекции в реальном времени и передовые возможности трансформерных архитектур TrOCR и Donut для качественного распознавания текста, включая сложные случаи и возможность работы без традиционных OCR-движков.

4.3 Гиперпараметры и стратегия обучения

Ниже представлены ключевые гиперпараметры и детали стратегии обучения, использованные для каждой из моделей в нашем проекте.

4.3.1 YOLOv12

Для обучения модели детекции текста YOLOv12 мы использовали следующие настройки:

- **Основные параметры модели и данных:**
 - Размер входного изображения: 640x640 пикселей.
 - Задача: детекция одного класса (`single_cls: true`).
- **Параметры процесса обучения:**
 - Количество эпох: 4.
 - Размер пакета (Batch size): автоматический подбор (`batch: -1`).
 - Оптимизатор: (`optimizer: SGD`).

- Начальная скорость обучения (Learning rate, `lr0`): 0.01.
- Конечный фактор скорости обучения (Learning rate factor, `lrf`): 0.01.
- Моментум: 0.937.
- Регуляризация (Weight decay): 0.0005.
- Эпохи разогрева (Warmup epochs): 3.0.
- Моментум на этапе разогрева: 0.8.
- Скорость обучения для смещений (bias) на этапе разогрева: 0.1.
- Использование смешанного режима точности (AMP): включено (`amp: true`).
- Завершение аугментации типа "mosaic" за N последних эпох (`close_mosaic`): 0 (т.е. mosaic не используется до конца).

- **Аугментации данных (во время обучения):**

- Автоматическая аугментация: `randaugment`.
- Изменение цветового пространства HSV: H (`hsv_h`: 0.015), S (`hsv_s`: 0.7), V (`hsv_v`: 0.4).
- Геометрические преобразования:
 - * Повороты (`degrees`): 0 (отключены в данном конфиге, но `randaugment` может применять).
 - * Сдвиг (`translate`): 0.1.
 - * Масштабирование (`scale`): 0.5.
 - * Сдвиг с наклоном (`shear`): 0.0.
 - * Перспектива (`perspective`): 0.0.
- Горизонтальное отражение (`fliplr`): вероятность 0.5.
- Вертикальное отражение (`flipud`): вероятность 0.0.
- Аугментация Mosaic: вероятность 1.0.
- Аугментация MixUp: вероятность 0.0 (отключена).
- Случайное стирание (`erasing`): вероятность 0.4.
- Copy-Paste аугментация: вероятность 0.0 (отключена).

- **Прочее:**

- Устройство для обучения: GPU 0 (`device: '0'`).
- Количество рабочих процессов для загрузки данных (`workers`): 8.
- Начальное значение для генератора случайных чисел (`seed`): 42.
- Детерминированный режим: включен (`deterministic: true`).

4.3.2 TrOCR

Для обучения модели распознавания текста TrOCR мы использовали следующие настройки:

- **Основные параметры модели и данных:**
 - Размер входного изображения: 384x384 пикселей.
- **Параметры процесса обучения:**
 - Количество эпох: 5.
 - Размер пакета (Batch size): 32.
 - Скорость обучения (Learning rate): 5e-5.
 - Точность вычислений (Precision): bf16.
- **Аугментации данных:**
 - Те же, что и для Donut
- **Оптимизации памяти/скорости:**
 - Заморозка энкодера: использовалась опция `-freeze_encoder`.
 - Использование Flash Attention: применялась опция `-flash_attention`.
 - Накопление градиента: применялась опция `-gradient_accumulation_steps`.

4.3.3 Donut

Для обучения модели распознавания текста Donut (дообучение на основе `AkaJackson/donut_rus`) мы использовали следующие настройки:

- **Основные параметры модели и данных:**
 - Исходная модель: `AkaJackson/donut_rus`.
 - Размер входного изображения: 384x384 пикселей.
 - Максимальная длина генерируемой последовательности: 64 токена.
 - Стартовый токен задачи: `<s_500k>`.
 - Конечный токен промпта: `<s_prompt>`.
- **Параметры процесса обучения:**
 - Количество эпох: 25.
 - Размер пакета (Batch size): 80.
 - Скорость обучения (Learning rate): 5e-5.

- Регуляризация (Weight decay): 0.005.
- Доля эпох для разогрева (Warmup ratio): 0.005.
- Шаги накопления градиента (Gradient Accumulation Steps): 16.
- Максимальная норма градиента (Max grad norm): 100.0.
- Точность вычислений (Precision): bf16.
- **Аугментации данных:**
 - Применение аугментаций: включено (`apply_augmentation: true`).
 - Вероятность применения аугментации: 0.3 (`augmentation_prob`).
 - Максимальный угол поворота: ± 8.0 градусов (`max_rotation`).
 - Диапазон яркости: [0.8, 1.2] (`brightness_min`, `brightness_max`).
 - Диапазон контрастности: [0.8, 1.2] (`contrast_min`, `contrast_max`).
 - Диапазон размытия (blur kernel size): [0, 2] (`blur_min`, `blur_max`).
 - Уровень шума: 0.05 (`noise_level`).
 - Диапазон резкости: [0.8, 1.5] (`sharpness_min`, `sharpness_max`).
- **Прочее:**
 - Устройство для обучения: cuda.
 - Количество рабочих процессов для загрузки данных (`num_workers`): 8.
 - Ранняя остановка (Early stopping): терпение 60 шагов (`early_stopping_patience`), порог 0.1 (`early_stopping_threshold`).

5 Реализация и обучение

5.1 Описание кода и окружения

5.1.1 Окружение и фреймворки

- **Python:** Версия 3.10 и выше.
- **GPU:** Рекомендуются NVIDIA GPU (например, A100 80GB) для эффективного обучения.
- **CUDA:** Версия 12.1 и выше.
- **Фреймворки:** PyTorch, Hugging Face Transformers, Ultralytics YOLO.

5.1.2 Структура кода и скрипты обучения

Обучение моделей YOLOv12, TrOCR и Donut производится с использованием соответствующих скриптов и Jupyter-ноутбуков, доступных в ветке *train* репозитория.

- **Ветка *train*:** Отвечает за обучение, оптимизацию и подготовку моделей компьютерного зрения. Содержит все необходимые скрипты, конфигурации и инструменты.
- **YOLOv12:** Обучение осуществляется с использованием Jupyter-ноутбуков в директории *train/notebooks*. Скрипты для обучения и инференса также находятся в директории *train_yolo*. Обученные модели экспортируются в формат TensorRT через `model.export(format='engine')`.
- **TrOCR:** Обучение производится с использованием скрипта *train_trocr/train.py*.
- **Donut:** Обучение осуществляется с использованием скрипта *train_donut/train.py*. Для Donut предусмотрены специальные скрипты для экспорта в TensorRT.

5.2 Процесс обучения

Точное время обучения зависит от используемого оборудования (GPU), размера датасета и выбранных гиперпараметров. Учитывая использование A100 GPU и оптимизированных фреймворков, обучение моделей занимает разумное время. Но, разумное в рамках практики глубокого обучения, мы модель не дообучили, полное обучение заняло примерно 14 часов для модели Donut и примерно 10 часов для модели Yolo.

5.2.1 YOLOv12

- **Метрики качества (во время обучения):** Динамика изменения ключевых метрик и потерь в процессе обучения модели YOLOv12 представлена на рисунке 5. Таблица 1 содержит сводку этих метрик для четырех эпох обучения.

5.2.2 TrOCR

Пример команды для обучения:

```
1 python train_trocr/train.py \  
2 --data_dir path/to/dataset \  
3 --output_dir ./trocr_output \  
4 --batch_size 32 \  
5 --precision bf16 \  
6 --image_size 384 384 \  
7 --learning_rate 5e-5 \  
8 --num_epochs 10
```

Листинг 1: Пример команды обучения TrOCR

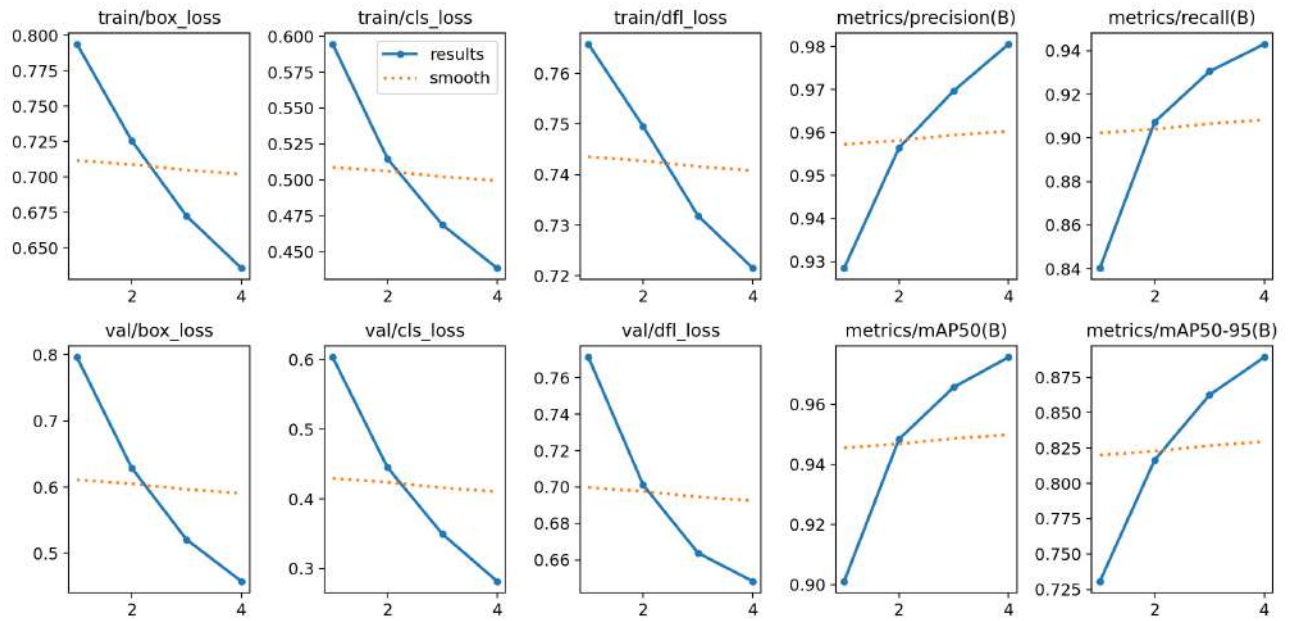


Рис. 5: Графики метрик и потерь обучения модели YOLOv12.

Табл. 1: Сводка метрик обучения YOLOv12 (первые 4 эпохи).

Метрика	Эпоха 1	Эпоха 2	Эпоха 3	Эпоха 4
Время эпохи (сек)	2125.79	2207.64	2155.75	2140.18
<i>Потери на обучающей выборке (Train Loss)</i>				
train/box_loss	0.79375	0.72553	0.67236	0.63558
train/cls_loss	0.59467	0.51476	0.4687	0.43846
train/dfl_loss	0.76579	0.74952	0.73185	0.7215
<i>Метрики на валидационной выборке (Validation Metrics)</i>				
metrics/precision(B)	0.92845	0.95642	0.96971	0.98056
metrics/recall(B)	0.84026	0.90733	0.93051	0.94302
metrics/mAP50(B)	0.90105	0.94836	0.96571	0.97566
metrics/mAP50-95(B)	0.73067	0.81653	0.86257	0.88925
<i>Потери на валидационной выборке (Validation Loss)</i>				
val/box_loss	0.79622	0.62871	0.52074	0.45762
val/cls_loss	0.60364	0.44523	0.34971	0.28156
val/dfl_loss	0.77125	0.70109	0.66379	0.64822
<i>Скорость обучения (Learning Rate)</i>				
lr/pg0	0.074136	0.048258	0.022375	0.010447
lr/pg1	0.003025	0.006049	0.009068	0.010447
lr/pg2	0.003025	0.006049	0.009068	0.010447

- **Метрики качества (во время обучения):** Динамика изменения потерь на обучающей выборке в процессе обучения модели TrOCR представлена на рисунке 6. Таблица 2 содержит сводку этих потерь, скорости обучения и времени на эпоху для первых пяти эпох.

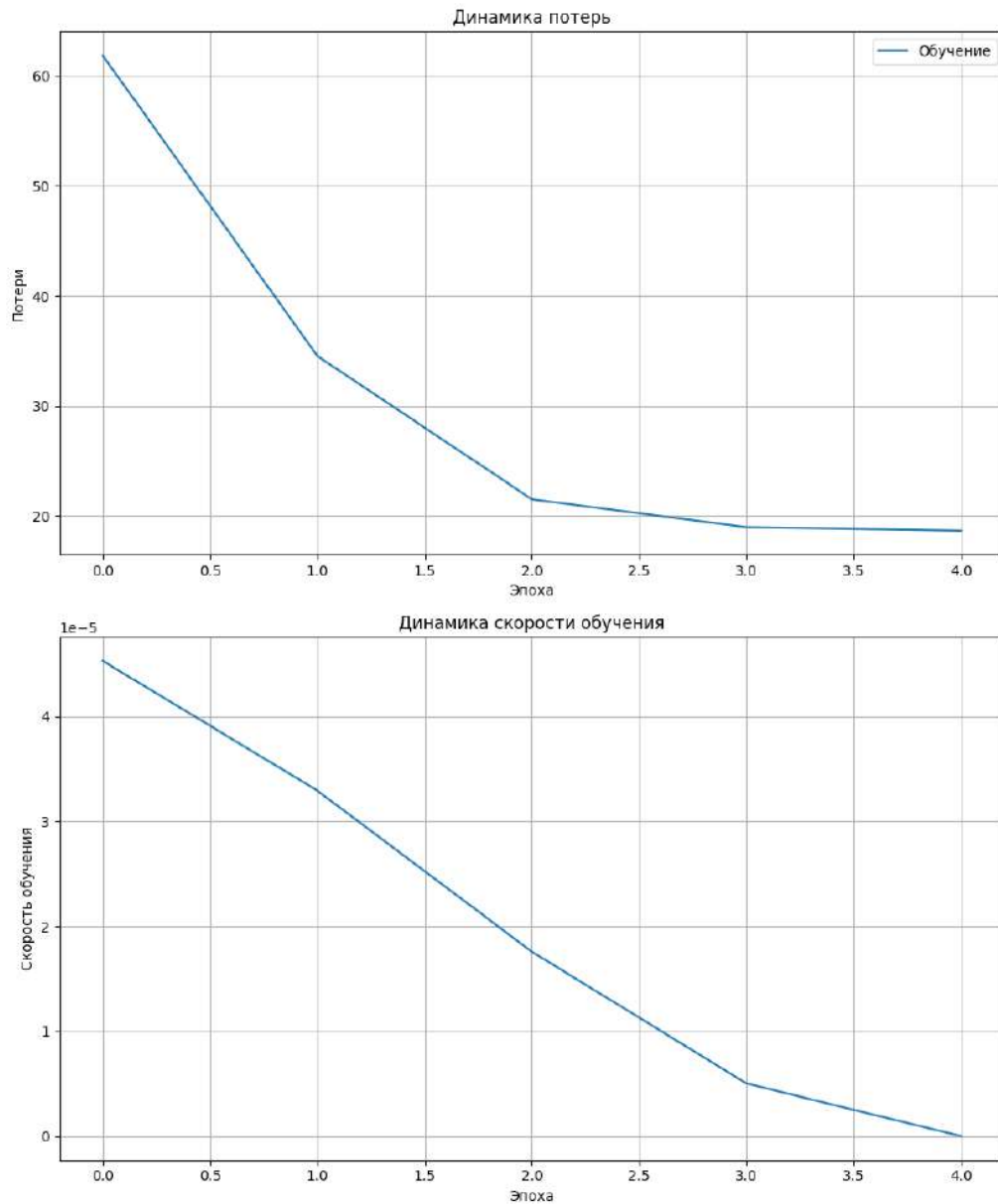


Рис. 6: График потерь обучения модели TrOCR (первые 5 эпох).

Табл. 2: Сводка метрик обучения TrOCR (первые 5 эпох).

Метрика / Эпоха	1	2	3	4	5
Train Loss	61.8813	34.5386	21.5036	18.9625	18.6446
Learning Rate	4.53e-05	3.29e-05	1.76e-05	5.02e-06	5.24e-09
Time per Epoch (s)	7139.92	7166.27	7161.35	7166.90	7168.94

5.2.3 Donut

Пример команды для обучения:

```
1 python train_donut/train.py \  
2 --data_dir path/to/documents \  
3 --output_dir ./donut_output \  
4 --batch_size 80 \  
5 --image_size 384 384 \  
6 --apply_augmentation \  
7 --num_epochs 30
```

Листинг 2: Пример команды обучения Donut

- **Метрики качества (во время обучения):** Динамика изменения потерь на обучающей и валидационной выборках в процессе обучения модели Donut представлена на рисунке 7. Таблица 3 содержит сводку этих потерь, скорости обучения и времени на эпоху для выбранных эпох. На 5-й эпохе наблюдается значительный скачок потерь и существенное уменьшение времени на эпоху, так как сначала Donut обучалась на синтетических данных, а затем тюнилась на реальных.

Табл. 3: Сводка метрик обучения Donut (выборочные эпохи).

Метрика / Эпоха	1	2	3	4	5	25
Train Loss	3.1420	0.9779	0.6399	0.5102	7.6620	1.7144
Val Loss	0.8649	0.4417	0.2957	0.2519	6.8244	1.8483
Learning Rate	4.54e-05	3.29e-05	1.74e-05	4.82e-06	1.25e-07	1.11e-07
Time per Epoch (s)	8797.71	8792.64	8793.67	8793.04	351.68	347.86

5.3 Метрики для детекции текста (YOLO)

Для оценки качества модели детекции текста YOLOv12 используются следующие стандартные метрики:

- **Precision (Точность):** Доля правильно обнаруженных текстовых областей среди всех предсказанных моделью. Высокая точность означает, что модель делает мало ложных срабатываний.
- **Recall (Полнота):** Доля правильно обнаруженных текстовых областей среди всех реально существующих текстовых областей в данных. Высокая полнота означает, что модель находит большинство текстовых объектов.
- **F1-score:** Гармоническое среднее Precision и Recall, предоставляющее сбалансированную оценку, особенно полезную при дисбалансе классов или когда важны и точность, и полнота. Рассчитывается как $2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$.

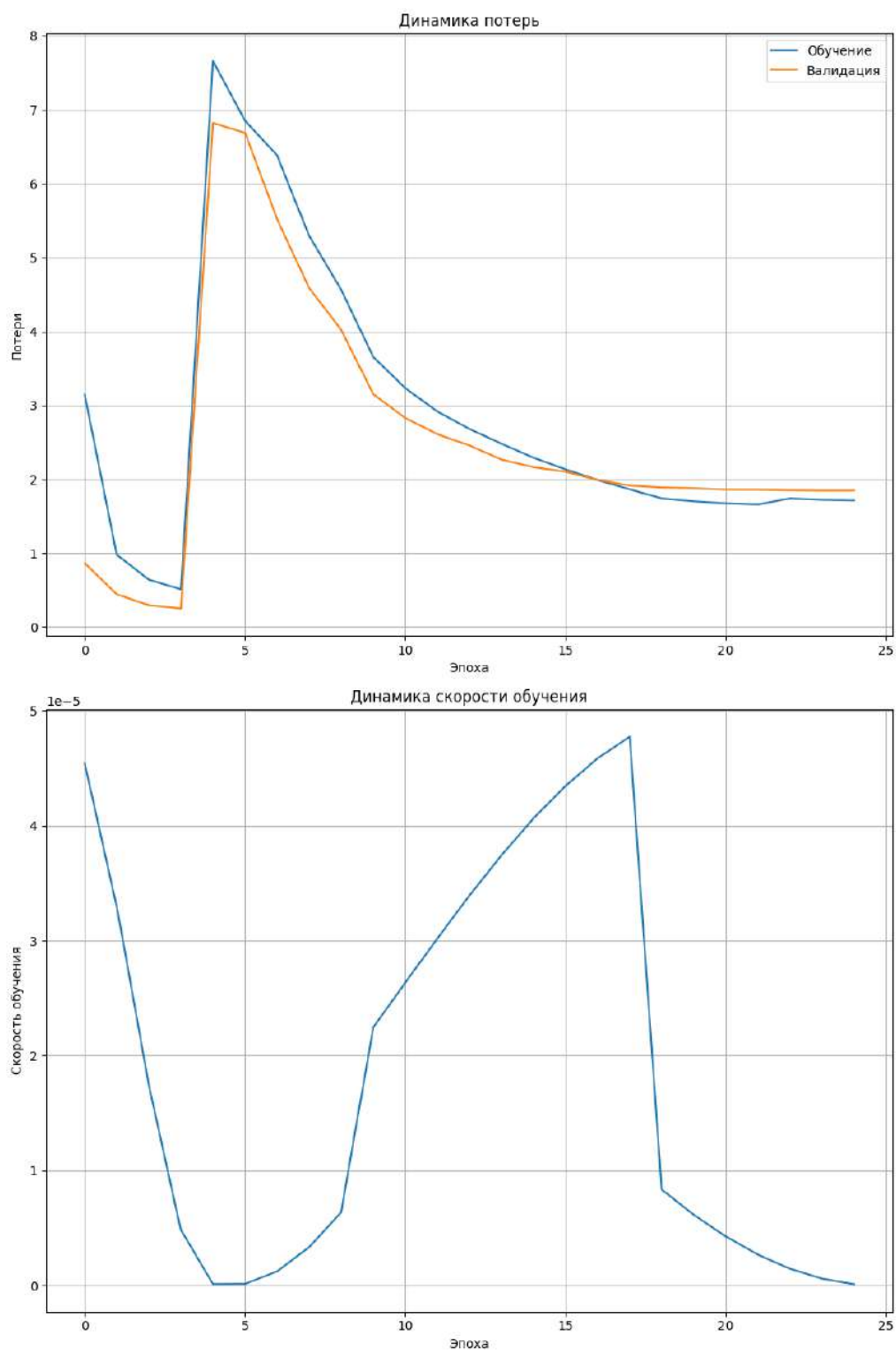


Рис. 7: График потерь обучения модели Donut (25 эпох).

- **mAP (mean Average Precision):** Среднее значение Average Precision (AP) по всем классам. В нашем случае, поскольку класс один ("текст"), mAP@0.5 означает AP при пороге IoU = 0.5, а mAP@0.5:0.95 означает среднее AP по диапазону порогов IoU от 0.5 до 0.95 с шагом 0.05. Это основная метрика для оценки общей производительности моделей детекции объектов.

- **IoU (Intersection over Union):** Метрика, измеряющая степень перекрытия предсказанной ограничивающей рамки с истинной. Рассчитывается как площадь пересечения рамок, деленная на площадь их объединения. Используется для определения, является ли предсказание верным (True Positive).

Визуализация результатов работы модели и кривых метрик представлена ниже.

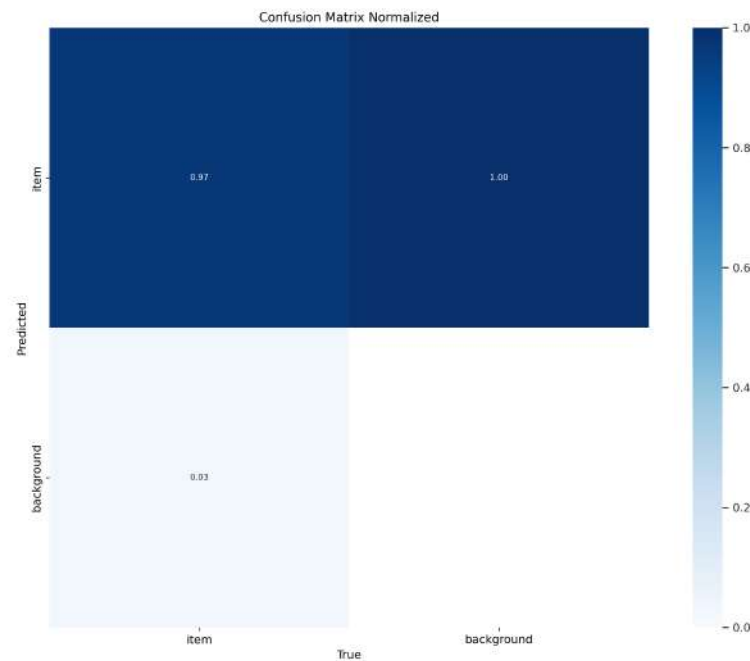


Рис. 8: Нормализованная матрица ошибок для модели детекции YOLOv12.

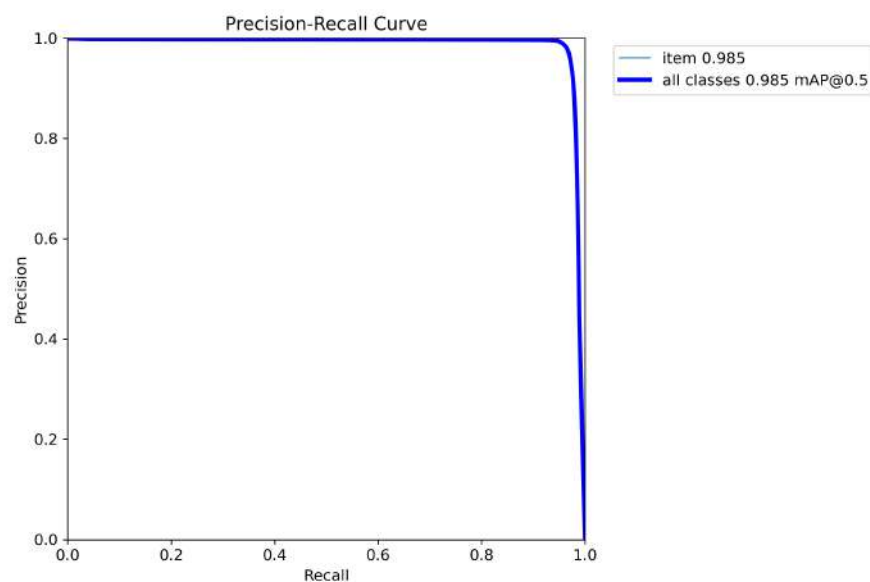
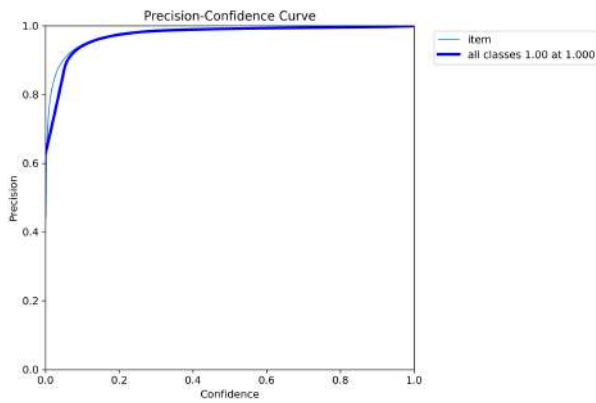
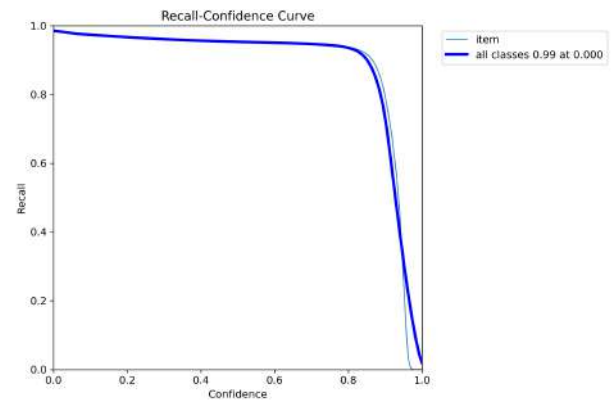


Рис. 9: Кривая Precision-Recall (Точность-Полнота) для модели YOLOv12.



(a) Кривая Precision.



(b) Кривая Recall.

Рис. 10: Кривые Precision и Recall в зависимости от порога уверенности (confidence) для модели YOLOv12.

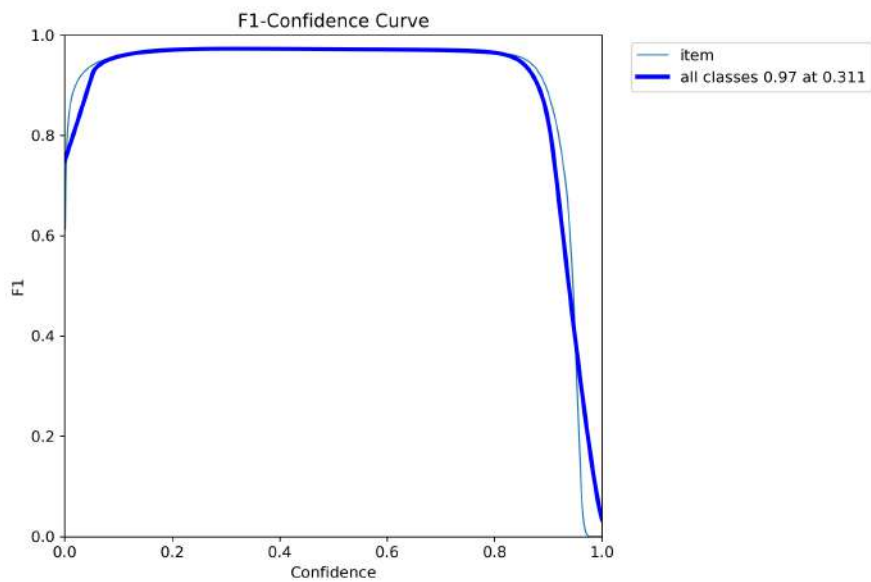
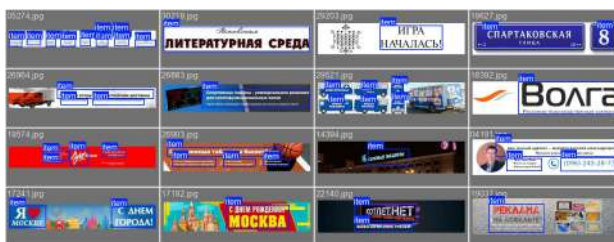


Рис. 11: Кривая F1-score в зависимости от порога уверенности (confidence) для модели YOLOv12.



(a) Пример разметки (ground truth) из валидационного набора.



(b) Предсказания модели YOLOv12 для соответствующего изображения.

Рис. 12: Визуальный пример работы модели YOLOv12: истинная разметка и предсказания модели на изображении из валидационного набора.

5.4 Метрики для распознавания текста (TrOCR, Donut)

Мы использовали метрику **Character Similarity Score** (Коэффициент символьного сходства). Эта метрика рассчитывается на основе нормализованного расстояния редактирования (edit distance), такого как расстояние Левенштейна. Нормализованное расстояние редактирования (`normalized_edit_distance`) представляет собой долю ошибок (0.0 для идентичных строк, 1.0 для полностью различных строк при одинаковой длине, или если одна из строк пуста). Соответственно, **Character Similarity Score** рассчитывается как $1.0 - \text{normalized_edit_distance}$. Таким образом:

- Значение **1.0** означает идеальное совпадение предсказанной строки с эталонной на уровне символов.
- Значение, близкое к **0.0**, указывает на очень низкое сходство между предсказанием и эталоном.

На рисунке 13 представлено распределение значений Character Similarity Score для результатов распознавания одной из наших моделей (Donut). Это позволяет визуально оценить, какая доля предсказаний имеет высокий уровень сходства с эталонами, а какая — низкий. Идеальное распределение было бы сконцентрировано вблизи значения 1.0.

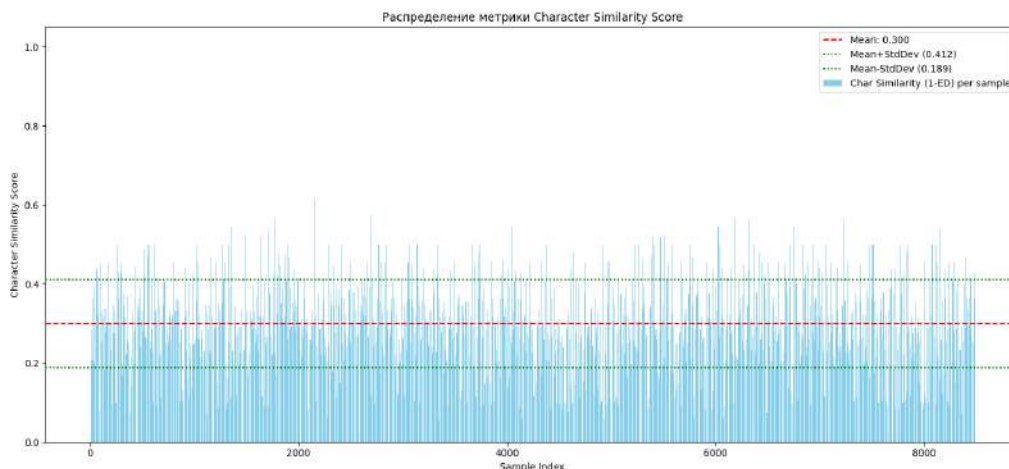


Рис. 13: Распределение коэффициента символьного сходства (Character Similarity Score) для результатов распознавания текста.

6 Развертывание и демонстрация

Развернутое решение FrameReader состоит из нескольких компонентов: бэкенд-сервера (ветка *server*), фронтенд-приложения (ветка *client*) и сервера для инференса моделей (ветка *triton — server*).

6.1 Использование в продакшене (Сервер инференса)

Для развертывания обученных моделей в продакшене используется NVIDIA Triton Inference Server в связке с Ray Serve. Такой подход обеспечивает высокую производительность, масштабируемость и гибкость инференса. Модели экспортируются в оптимизированный формат, такой как TensorRT engine, для достижения максимальной производительности.

Архитектура сервера инференса: Архитектура построена на следующих ключевых компонентах:

- **NVIDIA Triton Inference Server:** Открытая платформа для инференса, поддерживающая различные фреймворки (TensorFlow, PyTorch, ONNX Runtime, TensorRT) и обеспечивающая высокую пропускную способность и низкую задержку.
- **Ray Serve:** Библиотека для развертывания моделей и сервисов на Ray, которая легко интегрируется с Triton Server для управления инференсом.

Поддерживаемые модели и их реализация в Triton Server:

- **YOLO:** Используется для детекции текстовых областей (предположительно YOLOv12). Модель загружается в Triton Server в оптимизированном формате (например, TensorRT engine).
- **Donut:** Используется для оптического распознавания символов (OCR). Для этой модели реализован **кастомный движок TensorRT инференса**, что позволяет адаптировать стандартный инференс Donut для работы с TensorRT и достичь максимальной производительности.

Модели для Triton Server располагаются в директории *triton — server/models/*. Каждая модель имеет свою поддиректорию с файлом конфигурации *config.pbtxt* и версиями модели (например, 1/).

6.2 API и интерфейсы

6.2.1 API сервера инференса

Triton Server и Ray Serve предоставляют несколько эндпоинтов для взаимодействия:

- **Ray Serve API:** Эндпоинты для инференса моделей, такие как *http : //localhost : 8000/yolo* и *http : //localhost : 8000/donut*. Также доступен эндпоинт для проверки состояния *http : //localhost : 8000/health*.
- **Метрики и мониторинг:** Эндпоинт для метрик *http : //localhost : 8002/metrics* и дашборд Ray *http : //localhost : 8265*.

Пример запроса к Ray Serve для инференса YOLO:

```

1  import requests
2  import base64
3
4  with open("image.jpg", "rb") as f:
5      image_data = base64.b64encode(f.read()).decode()
6
7      response = requests.post(
8          "http://localhost:8000/yolo",
9          json={"image": image_data}
10         )
11
12     print(response.json())

```

Листинг 3: Пример запроса к Ray Serve для YOLO

Поддерживается как основной (main) API, так и потоковый (streaming) API для получения результатов обработки.

6.2.2 API Бэкенда

Бэкенд-сервер (ветка *server*) предоставляет REST API и WebSocket API для взаимодействия с клиентским приложением.

- **REST API:** Используется для управления сессиями, например, создание сессии через POST-запрос на *http://localhost:8000/api/video_sessions*.
- **WebSocket API:** Используется для потоковой передачи аннотаций видео и получения результатов в реальном времени, например, через эндпоинт *ws://localhost:8000/ws/video_recognition/session_id*.

6.3 Компоненты системы

6.3.1 Бэкенд (Server)

Ветка *server* является основным бэкендом проекта, реализующим логику работы с базой данных, трекинг видео, распознавание текста и интеграцию с Triton Server.

Основные функции и архитектура бэкенда:

- **Трекинг видео:** Обработка видеопотоков для детекции текстовых областей.
- **Распознавание текста (OCR):** Извлечение и распознавание текста из выделенных областей.
- **Управление данными:** Хранение информации о пользователях, сессиях обработки видео и результатах распознавания в базе данных MySQL.

- **API:** Предоставление RESTful API и WebSocket API на основе FastAPI.
- **Интеграция с Triton Server** для выполнения моделей.

Структура базы данных: MySQL используется для хранения данных о пользователях, сессиях и результатах. Структура включает таблицы для сессий видео (*video_sessions*) и распознанных текстах (*annotation_frame*). Пример таблицы для пользователей:

```
1 CREATE TABLE IF NOT EXISTS users (  
2 id INT AUTO_INCREMENT PRIMARY KEY,  
3 username VARCHAR(255) NOT NULL UNIQUE,  
4 email VARCHAR(255) NOT NULL UNIQUE,  
5 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
6 );
```

Листинг 4: Пример таблицы для пользователей в MySQL

6.4 Демонстрация

Демо приложения

7 Выводы и перспективы

7.1 Основные результаты работы

Проект FrameReader демонстрирует комплексный подход к задаче извлечения и распознавания текста из видео. Разработанное решение объединяет передовые модели компьютерного зрения (YOLO, TrOCR, Donut) с эффективной инфраструктурой развертывания (Triton Server, Ray Serve) и удобным пользовательским интерфейсом. Это позволяет автоматизировать процесс анализа видеоконтента и получать структурированную текстовую информацию.

7.2 Нерешенные проблемы

Несмотря на достигнутые результаты, существуют аспекты, требующие дальнейшего внимания:

- Точность распознавания текста недостаточна хороша и может варьироваться для очень специфичных шрифтов, низкого качества видео или сложных условий съемки.
- Обработка рукописного текста в видео на данный момент не реализована.
- Оптимизация производительности для обработки видеопотоков сверхвысокого разрешения в реальном времени может потребовать дополнительных исследований.

7.3 Планы масштабирования

- **Улучшение точности OCR:** Дальнейшее обучение моделей на более разнообразных и специфичных датасетах для русского языка, а также исследование новых архитектур.
- **Обработка рукописного текста:** Расширение функционала для распознавания рукописного текста в видео.
- **Оптимизация производительности:** Дальнейшая оптимизация моделей и инфраструктуры для обработки видео в реальном времени с еще большей скоростью.
- **Расширение функционала:** Добавление возможностей для поиска по распознанному тексту, аннотирования видео, экспорта результатов в различные форматы.
- **Мультиязычность:** Поддержка распознавания текста на нескольких языках.